

Campaign POC — One Table, End To End, On Azure

Date: 2026-08-01 Scope: **Campaign only**. Prove the whole pipeline on the smallest object before touching the other 8.
Companion doc: [azure_migration_plan.md](#) — the full multi-service plan and trade-offs. **This POC implements that plan for one object**; every Azure tool used here is described and justified in the plan (see its \$0.6 and \$4).

Why Campaign

- Smallest object: ~41K rows (~50 MB). Cheap and fast to run many times.
- Already the most developed: raw table, dedup view, materialized latest, and 19 DQ rules exist.
- If the full flow (ingest -> stage -> check -> clean -> ready to push back) works here, it repeats for every other object by changing names only.

The Invariant This POC Must Prove

Raw may contain duplicates. Staging must not. Final (clean, push-ready) must not start until it is safe.

Enforced as three hard gates:

1. **Raw gate**: append-only. Duplicates and history are allowed and expected.
2. **Staging gate**: exactly one row per Campaign `Id` (latest by `SystemModstamp`), deleted rows excluded.
3. **Final gate**: the clean / push-ready set is only built when **0 CRITICAL DQ failures** remain for the rows in scope. If any critical failure exists, final does **not** start.

And the whole thing is **incremental** and **restartable**: at any point we can ask the control tables "where are we?" and re-derive state from the start without guessing.

Part A — What Is Currently Built For Campaign

Stage	Object today	Notes
Ingest	<code>raw.salesforce_campaign</code>	Bulk API 2.0 via <code>python.py</code> ; ~41K rows incl. 529 duplicates; all <code>NVARCHAR(MAX)</code>
Control	<code>ctl.etl_run_control</code> , <code>ctl.watermark_control</code> , <code>ctl.object_registry</code> , <code>ctl.loaded_salesforce_campaign_ids</code>	run log, watermark cursor, metadata, resume ids
Staging (view)	<code>staging.vw_campaign_latest</code>	<code>ROW_NUMBER()</code> PARTITION BY <code>Id</code> ORDER BY <code>SystemModstamp</code> DESC, excludes <code>IsDeleted</code>
Staging (table)	<code>staging.campaign_latest</code>	materialized 40,775 unique rows + <code>staging_is_duplicate</code> , <code>staging_duplicate_count</code> , <code>staging_created_at</code>
Checks	19 rules CAM-001..CAM-017 + CAM-URL-001	see <code>mohey_work/Tables/campaign/campaign_staging_dq_PROD.sql</code>
Checks (framework)	<code>dq.dq_rule_catalog</code> + watermark-incremental runner	reruns only rows past the DQ cursor
Clean	not built yet	this POC adds it
Push-ready / Writeback	designed only	this POC adds a staged, approval-gated set

Verified evidence (2026-07-29 run): raw non-deleted 41,304 -> staging 40,775 (529 duplicates removed); 0 CRITICAL failures, 9 HIGH, ~4,901 MEDIUM, ~13,318 LOW. `CAM-004` flags **In Progress** (31,280 rows) — a stakeholder decision, not a defect.

Part B — What We Will Do (POC target)

Add the two missing stages and make the whole chain incremental + checkpointed in Azure:

```
Salesforce Campaign
-> [1 INGEST]   raw.salesforce_campaign      (append-only, duplicates OK)
-> [2 STAGE]    staging.campaign_latest      (1 row per Id, no duplicates)
-> [3 CHECK]    dq.* rules over staging      (flag violations, block criticals)
-> [4 CLEAN]    staging.campaign_clean       (normalized, only rows that pass gate)
-> [5 PUSH-READY] writeback.campaign_pending (approved corrections queued for Salesforce)
```

Each arrow writes a checkpoint into a control/state table so any run can resume and any auditor can re-derive "where are we" from the start.

Part C — Stage By Stage

Stage 1 — Ingest (raw, duplicates allowed)

- Runs `python.py` for Campaign in one of three modes chosen automatically:
 - Full** — first load, table empty.
 - Incremental** — `WHERE SystemModstamp > last_watermark AND <= now.`
 - ResumeMissing** — last run not `Succeeded`; loads only `Id NOT IN ctl.loaded_salesforce_campaign_ids.`
- Raw is **append-only**. We never dedupe here. Re-runs can legitimately create duplicate `Ids` (same record modified twice, or an overlap window). That is fine — this is history.

Checkpoint written: a row in `ctl.etl_run_control` (status, rows) and the advanced `ctl.watermark_control.last_watermark_value.`

Check from the start at any time:

```
-- Where is ingest right now?
SELECT TOP 5 run_id, load_type, status, rows_extracted, rows_loaded, start_time, end_time
FROM ctl.etl_run_control
WHERE object_name = 'Campaign'
ORDER BY run_id DESC;

SELECT object_name, last_watermark_value, last_success_run_id
FROM ctl.watermark_control
WHERE object_name = 'Campaign';

-- Raw is allowed to have duplicates:
SELECT COUNT(*) AS raw_rows,
       COUNT(DISTINCT CONVERT(VARCHAR(18), Id)) AS distinct_ids
FROM raw.salesforce_campaign;  -- raw_rows >= distinct_ids is expected
```

Stage 2 — Stage (latest, no duplicates) — GATE

- Rebuild `staging.campaign_latest` from `staging.vw_campaign_latest` (or refresh SP).
- Dedup rule: keep rank 1 per `Id` ordered by `SystemModstamp DESC`, tie-break `_etl_run_id DESC`; exclude soft-deleted rows.
- Gate 2 (must pass)**: staging has exactly one row per `Id` and zero duplicates.

```
-- GATE 2: staging must be unique per Id. This must return 0 rows.
SELECT CONVERT(VARCHAR(18), Id) AS Id, COUNT(*) AS n
FROM staging.campaign_latest
GROUP BY CONVERT(VARCHAR(18), Id)
HAVING COUNT(*) > 1;

-- Evidence of dedup: how many raw duplicates were collapsed
SELECT
  (SELECT COUNT(*) FROM raw.salesforce_campaign
   WHERE COALESCE(LOWER(LTRIM(RTRIM(CONVERT(NVARCHAR(20), IsDeleted)))), 'false')
   NOT IN ('true', '1', 'yes', 'y')) AS raw_non_deleted,
  (SELECT COUNT(*) FROM staging.campaign_latest) AS staging_rows;
```

If Gate 2 fails, **stop**. Do not proceed to checks or clean.

Stage 3 — Check (data quality) — GATE

- Apply the 19 Campaign rules over `staging.campaign_latest`, writing violations to the `dq` layer (`dq.dq_exceptions` / the campaign exceptions table), tagged by rule id and severity.
- Incremental: the DQ runner only scans rows with `SystemModstamp > dq_cursor`, ordered by watermark, in bounded batches, then advances the cursor. A full re-scan is available by resetting the cursor to NULL.
- **Gate 3 (must pass to start clean/final): 0 CRITICAL** failures for rows in scope. HIGH/MEDIUM/LOW are recorded but do not block — they route to review.

```
-- GATE 3: no critical failures allowed before Clean/Final may start.
SELECT severity, COUNT(*) AS violations, COUNT(DISTINCT record_id) AS affected_campaigns
FROM dq.dq_exceptions
WHERE object_name = 'Campaign'
GROUP BY severity
ORDER BY CASE severity WHEN 'CRITICAL' THEN 1 WHEN 'HIGH' THEN 2
                  WHEN 'MEDIUM' THEN 3 ELSE 4 END;

-- Must return 0 for Final to begin:
SELECT COUNT(*) AS critical_open
FROM dq.dq_exceptions
WHERE object_name = 'Campaign' AND severity = 'CRITICAL';
```

Where is the DQ runner? Inspect the rule execution state (watermark cursor per rule):

```
SELECT r.rule_id, r.check_name, s.last_source_watermark_value,
       s.last_run_status, s.last_run_rows_checked, s.last_run_failed_count
FROM dq.dq_rule_catalog r
JOIN dq.rule_execution_state s ON s.rule_id = r.rule_id
WHERE r.object_name = 'Campaign'
ORDER BY r.rule_id;
```

Stage 4 — Clean (normalized, gated build)

- Build `staging.campaign_clean` **only if Gate 3 passed** (0 critical).
- "Clean" here is intentionally simple and safe (no business overwrites without approval):
 - trim whitespace on key text fields,
 - normalize `CurrencyIsoCode` to upper case,
 - cast dates via `TRY_CONVERT` (invalid -> NULL, and already flagged in DQ),
 - normalize `IsDeleted/IsActive` boolean text to `0/1`,

- carry the `Id` and `SystemModstamp` unchanged (they are keys/lineage).
- Rows that fail non-critical rules are still included but **tagged** (`clean_flag`, `review_reason`) so nothing is silently dropped.
- **Final cannot start until this gate holds:** Clean is only populated for the in-scope, non-critical set.

```
-- Final gate check before/after building clean:
-- 1) unique per Id (inherited from staging)
SELECT CONVERT(VARCHAR(18), Id) AS Id, COUNT(*) n
FROM staging.campaign_clean GROUP BY CONVERT(VARCHAR(18), Id) HAVING COUNT(*) > 1; -- expect 0

-- 2) no critical-failing Id leaked into clean
SELECT COUNT(*) AS bad
FROM staging.campaign_clean c
WHERE EXISTS (SELECT 1 FROM dq.dq_exceptions e
              WHERE e.object_name='Campaign' AND e.severity='CRITICAL'
                    AND e.record_id = CONVERT(VARCHAR(18), c.Id)); -- expect 0
```

Stage 5 — Push-ready (writeback queue, approval gated)

- Corrections that a reviewer approves are queued in `writeback.campaign_pending` with: `Id`, field, old value, new value, rule/reason, approver, status (`Pending/Approved/Pushed`).
- Nothing is written to Salesforce automatically. A separate, approved job pushes only `Approved` rows back via the Salesforce API, then marks them `Pushed` with a timestamp and result.
- This is the "ready to push into Salesforce" endpoint of the POC. Actual push is the last, guarded step.

```
-- What is ready to push, and what already went back?
SELECT status, COUNT(*) FROM writeback.campaign_pending
WHERE object_name = 'Campaign' GROUP BY status;
```

Part D — Incremental + "Check From The Start At Any Point"

Every stage has a cursor/log so the pipeline is restartable and auditable:

Stage	Cursor / checkpoint	Reset-to-start action
Ingest	<code>ctl.watermark_control.last_watermark_value</code> + <code>ctl.etl_run_control</code>	set watermark NULL -> next run is Full from the start
Ingest resume	<code>ctl.loaded_salesforce_campaign_ids</code>	drives ResumeMissing after a crash
Stage	rebuild is deterministic from raw	re-run refresh; output is idempotent
Check	<code>dq.rule_execution_state.last_source_watermark_value</code> (per rule)	set NULL -> full re-scan from the start
Clean	derived from staging + passing DQ	drop/rebuild; idempotent
Push-ready	<code>writeback.campaign_pending.status</code>	approvals are the audit trail

One combined "where are we" query answers state at any moment:

```
SELECT 'ingest' AS stage, CAST(last_watermark_value AS VARCHAR(30)) AS cursor_value
FROM ctl.watermark_control WHERE object_name='Campaign'
UNION ALL
```

```
SELECT 'raw_rows', CAST(COUNT(*) AS VARCHAR(30)) FROM raw.salesforce_campaign
UNION ALL
SELECT 'staging_rows', CAST(COUNT(*) AS VARCHAR(30)) FROM staging.campaign_latest
UNION ALL
SELECT 'critical_open', CAST(COUNT(*) AS VARCHAR(30))
FROM dq.dq_exceptions WHERE object_name='Campaign' AND severity='CRITICAL';
```

Because each stage is idempotent and cursor-driven, a run can stop anywhere and safely resume — and an auditor can reconstruct the exact position from these tables without reading logs.

Part E — The POC (Fully On Azure)

USING **Azure Portal** (portal.azure.com) and its in-browser tools (Query editor, ADF Studio, Cloud Shell).

Ingest uses **ADF's native Salesforce connector** (no container image to build), which is the easiest fully-portal path for a 41K-row object. The container/`python.py` path from [azure_migration_plan.md](#) stays the production option, but it is not needed to prove this POC.

How ADF runs this — what runs Python, where the logs are, and writing to `ctl`

How Data Factory works. ADF is a managed orchestrator. A **pipeline** is an ordered set of **activities** (Lookup, Copy, Script, Stored procedure, If/Until). Activities reach systems through **linked services** (connection + auth) and **datasets** (the table/file shape), and execute on an **Integration Runtime (IR)** — the serverless **Azure AutoResolve IR** (Microsoft-managed, nothing to install) runs Copy and pipeline activities; a **Self-hosted IR** is only needed to reach on-prem/private networks. A **trigger** (schedule / tumbling-window / event) starts a pipeline; for the POC just use **Debug** or **Trigger now**.

What runs Python. In this portal-first POC, **no Python runs** — ADF's native Salesforce **Copy** does the ingest and every transform is **T-SQL** (Script / Stored procedure activities) inside Azure SQL. If you keep `python.py` (Full/Incremental/ResumeMissing), it runs as a **container** — an **Azure Container App Job** or an ADF **Custom activity on Azure Batch** — that ADF *triggers*. In production the plan runs the same Python via **Apache NiFi** (plan §4.2). So: ADF/NiFi orchestrates; a container/host runs Python.

Where the logs are + how to see them.

- **ADF run history:** ADF Studio → **Monitor** → *Pipeline runs / Activity runs*. Each activity shows input, output, duration, **rows copied**, and the error. First place to look.
- **Persistent logs + alerts:** ADF → **Diagnostic settings** → send `PipelineRuns / ActivityRuns` to a **Log Analytics** workspace, then query with KQL, e.g. `ADFActivityRun | where Status == 'Failed'`.
- **Copy row-level logs:** the Copy activity's **Logging** setting writes skipped-row logs to the storage account (`sfdwpocsa`).
- **Your business log:** the `ctl.etl_run_control / ctl.watermark_control` rows *are* the pipeline's audit trail — query them in **Query editor** (see the checks in Part C).

Writing to the `ctl` schema from ADF (yes, you can). Use a **Stored procedure** or **Script** activity on the Azure SQL linked service. Best practice: wrap the writes in a proc and let ADF pass parameters:

```
-- one-time: a proc ADF calls after each load
CREATE OR ALTER PROCEDURE ctl.save_campaign_run
    @load_type varchar(20), @status varchar(20), @rows int, @new_wm varchar(33), @start datetime2
AS
BEGIN
    INSERT INTO ctl.etl_run_control (object_name, load_type, status, rows_loaded, start_time,
end_time)
    VALUES ('Campaign', @load_type, @status, @rows, @start, SYSUTCDATETIME());
    UPDATE ctl.watermark_control
        SET last_watermark_value = @new_wm, last_success_run_id = SCOPE_IDENTITY()
```

```
WHERE object_name = 'Campaign';
END
```

Then add a **Stored procedure activity** `save_wm` and map its parameters from pipeline expressions:

`@activity('copy_campaign').output.rowsCopied`, `@activity('new_wm').output.firstRow.wm`, `@pipeline().TriggerTime`. That is exactly the `save_wm` step in Stage 1 below.

Resource names used below (create them once): resource group `rg-sfdw-poc`, SQL server `sfdw-poc-sql`, database `SalesforceDW`, Key Vault `kv-sfdw-poc`, storage `sfdwpocsa`, data factory `sfdw-poc-adf` (region UK South because the data is GBP).

Prerequisite A — Provision (Portal, click-through)

1. **Resource group** — Portal -> Resource groups -> Create -> `rg-sfdw-poc`, region UK South.
2. **Azure SQL Database** — Create a resource -> SQL Database -> new server `sfdw-poc-sql` (Authentication: **Microsoft Entra-only**, set yourself as admin), database `SalesforceDW`, Compute+storage: **Serverless**, auto-pause 1 hour. Networking tab: turn on **Allow Azure services and resources to access this server**.
3. **Key Vault** — Create -> `kv-sfdw-poc` (RBAC permission model). Then Secrets -> Generate/Import -> `salesforce-client-secret` and paste the value.
4. **Storage (ADLS Gen2)** — Create -> Storage account `sfdwpocsa`, Advanced tab: enable **Hierarchical namespace**. After create: Containers -> + Container -> `raw`.
5. **Data Factory** — Create -> Data Factory `sfdw-poc-adf`, UK South. Open **Azure Data Factory Studio**.
6. **Identity + access** — the data factory gets a **system-assigned managed identity** automatically. Grant it access:
 - Key Vault -> Access control (IAM) -> Add role assignment -> **Key Vault Secrets User** -> the ADF identity.
 - Storage account -> IAM -> **Storage Blob Data Contributor** -> the ADF identity.
 - In Azure SQL (see Prereq B), add the ADF identity as a database user.

Prerequisite B — Deploy the database objects (Portal Query editor)

Open the database `SalesforceDW` -> **Query editor (preview)** -> sign in with Entra ID. It runs in the browser (on Azure). Paste and run each script's contents in this order (skip `00_database.sql` — the DB already exists):

1. `database/00_schemas.sql`
2. every file in `database/raw/`, then `database/ctl/`, then `database/dq/`, then `database/staging/`
3. `mohey_work/DQ Frame work/DQ_framework_creation_incremental.sql`
4. `mohey_work/DQ Frame work/DQ_framework_seed_checks_from_existing_prod.sql`

Then grant the ADF managed identity access (run once in Query editor):

```
CREATE USER [sfdw-poc-adf] FROM EXTERNAL PROVIDER; -- ADF managed identity name
ALTER ROLE db_owner ADD MEMBER [sfdw-poc-adf];
```

Tip: if pasting many files is tedious, open **Cloud Shell** (top bar >_), `git clone` the repo, and run `sqlcmd -S sfdw-poc-sql.database.windows.net -d SalesforceDW -G -N -C -b -i <file>`. Cloud Shell is still Azure, not your PC.

Stage 1 — Ingest: test BOTH paths (A = ADF Copy, B = container Python)

The POC deliberately proves **two ingest paths** into the *same* `raw.salesforce_campaign`, so Stages 2–5 (stage → check → clean) are identical whichever ran. Run **Path A**, check the counts; then run **Path B** into the same raw table and confirm it behaves the same — append-only, watermark advances, and ResumeMissing recovers a crash. Both record their run in `ctl.etl_run_control`.

Before Azure: validate the SQL side (the clean/fix build and the `ctl` logging proc) locally with `campaign_poc_local_test.sql`. The two ingest paths themselves need Azure.

Path A — ADF native Salesforce Copy (portal-first, no code to host)

This path uses the **ADF Salesforce (V2) connector** — Microsoft Learn: [Copy data from and to Salesforce](#). It talks to Salesforce **Bulk API 2.0** natively (the same API our Python uses), so there is no code to host.

Is it better than `python.py`? For standard ingest, yes (small compare):

- **Less to build/run** — managed connector, no image or host; built-in retry + monitoring.
- **Bulk API 2.0 built in** — issues the Bulk query for you; Full = no filter, Incremental = `WHERE SystemModstamp > @watermark`.
- **Same "args"** — Full vs Incremental is just the source query/watermark, mirroring Python's `--mode`.
- **Where Python still wins** — our custom **ResumeMissing** crash-recovery (loaded-ids) is not native; the connector would re-query the window instead.
- **Verdict** — use the connector for plain Full/Incremental (less code/ops); keep Python where ResumeMissing matters.

In **ADF Studio** build the linked services and the incremental copy. No code on your PC.

1. **Linked services** (Manage -> Linked services -> New):
 - **Salesforce (V2)** — auth via **Connected App / OAuth**, set the **API version**, Bulk API 2.0; put the client secret as a **Key Vault reference** to `kv-sfdw-poc/salesforce-client-secret` (never typed in ADF). Setup steps + supported properties: [connector docs](#).
 - **Azure SQL Database** — server `sfdw-poc-sql`, db `SalesforceDW`, auth **System-assigned managed identity**.
2. **Incremental copy pipeline** `pl_campaign_ingest` (this is the watermark pattern that maps to `ctl.watermark_control`, so raw stays append-only and duplicates are allowed):
 - **Lookup** `old_wm`: `SELECT last_watermark_value FROM ctl.watermark_control WHERE object_name='Campaign'`
 - **Lookup** `new_wm`: `SELECT CONVERT(VARCHAR(33), SYSUTCDATETIME(), 126) AS wm`
 - **Copy** `copy_campaign`: source = Salesforce query `SELECT ... FROM Campaign WHERE SystemModstamp > @{{activity('old_wm').output.firstRow.last_watermark_value}} AND SystemModstamp <= @{{activity('new_wm').output.firstRow.wm}}`; sink = `raw.salesforce_campaign`, **Insert** (append, no upsert — duplicates OK in raw).
 - **Stored procedure** `save_wm`: call `ctl.save_campaign_run` to log the run and advance the watermark (parameters mapped from pipeline expressions — see Part E primer). First run has NULL watermark = Full from start.
3. **Run** — prove **Full** (empty table / NULL watermark = no filter) then **Incremental** (watermark filter): the two runs that mirror Python's `--mode Full/Incremental`, using the same query/watermark `args`. Debug or Trigger now; because the watermark is stored, re-runs are incremental and restartable from the last committed point (the "check from the start at any point" property is preserved).

Verify Path A (Query editor):

```
SELECT TOP 3 run_id, load_type, status, rows_loaded
FROM ctl.etl_run_control WHERE object_name='Campaign' ORDER BY run_id DESC;

SELECT COUNT(*) raw_rows, COUNT(DISTINCT CONVERT(VARCHAR(18),Id)) distinct_ids
FROM raw.salesforce_campaign; -- raw_rows >= distinct_ids is expected (duplicates allowed)
```

Path B — `python.py` as an Azure Container App Job (keeps Full/Incremental/ResumeMissing)

This runs your **existing** Python unchanged, so all three modes are proven end-to-end.

1. **Build & push the image** (Azure Cloud Shell — still Azure, not your PC):

```
az acr create -g rg-sfdw-poc -n sfdwpocacr --sku Basic
az acr build -r sfdwpocacr -t sfdw/ingest:poc . # builds from the repo's Dockerfile
```

2. **Create the Container App Job** (Portal → Container App Jobs → Create, *Manual* trigger):

- Image `sfdwpocacr.azurecr.io/sfdw/ingest:poc`; **system-assigned managed identity** on.
 - Command/args: `python python.py --object Campaign --mode Incremental`.
 - Env: SQL server/db + Salesforce client id; the **secret** is a **Key Vault reference** to `kv-sfdw-poc/salesforce-client-secret` (never inline).
3. **Grant the job identity** the same access as ADF: Key Vault **Secrets User**, and a SQL user (`CREATE USER [job-campaign-ingest] FROM EXTERNAL PROVIDER; ALTER ROLE db_owner ADD MEMBER [job-campaign-ingest];`).
 4. **Run each mode** (Start the job, changing `--mode`):
 - `--mode Full` on an empty table (first load, from the start),
 - `--mode Incremental` (only `SystemModstamp > watermark`),
 - `--mode ResumeMissing` after you kill a run mid-way (loads only ids not in `ctl.loaded_salesforce_campaign_ids`).
 5. **Container logs:** Container App Job → *Execution history* → console logs, or Log Analytics table `ContainerAppConsoleLogs_CL`. The run also lands in `ctl.etl_run_control`, like Path A.

Compare A vs B (both must agree)

Both land append-only into the *same* raw table and write ctl rows, so one query compares them:

```
-- Runs recorded by each path, newest first
SELECT run_id, object_name, load_type, status, rows_loaded, start_time, end_time
FROM ctl.etl_run_control WHERE object_name='Campaign' ORDER BY run_id DESC;

-- Raw is still append-only after both paths ran (duplicates OK, distinct ids stable)
SELECT COUNT(*) raw_rows, COUNT(DISTINCT CONVERT(VARCHAR(18),Id)) distinct_ids
FROM raw.salesforce_campaign;
```

What it proves	Path A (ADF Copy)	Path B (container Python)
Full / Incremental	✅ watermark query	✅ <code>--mode Full/Incremental</code>
ResumeMissing (crash recovery)	⚠️ not native (re-query the window)	✅ loaded-ids table
Code to host	none	container image
Same raw + ctl result	✅	✅

Expectation: equal row counts and an append-only raw after each — the only difference is *how* the rows arrived. Path B additionally proves **ResumeMissing**, which ADF Copy does not do natively.

Stage 2 — Stage (dedup) as an ADF Script activity

Add a **Script activity** against the Azure SQL linked service (or run it in Query editor):

```
EXEC ctl.refresh_latest_views; -- rebuilds staging.vw_campaign_latest (1 row per Id)
```

Gate 2 — add a **Lookup** activity; fail the pipeline if it returns > 0:

```
SELECT COUNT(*) AS dup_ids FROM (
  SELECT CONVERT(VARCHAR(18),Id) Id FROM staging.campaign_latest
  GROUP BY CONVERT(VARCHAR(18),Id) HAVING COUNT(*) > 1
) x; -- must be 0
```

Stage 3 — Check (incremental DQ) as an ADF Stored Procedure activity

```
EXEC dq.run_incremental_catalog_rules
    @ObjectNameFilter='Campaign', @MaxRowsPerRule=100000, @MaxExceptionsPerRule=500;
```

Gate 3 — **Lookup**: fail the pipeline if critical > 0:

```
SELECT COUNT(*) AS critical_open
FROM dq.dq_exceptions WHERE object_name='Campaign' AND severity='CRITICAL'; -- must be 0
```

Full re-scan from the start (reset the per-rule cursor, then re-run Stage 3):

```
UPDATE s SET s.last_source_watermark_value=NULL
FROM dq.rule_execution_state s
JOIN dq.dq_rule_catalog r ON r.rule_id=s.rule_id
WHERE r.object_name='Campaign';
```

Stage 4 — **Clean**: fix 1–2 checks, skip the rest (the part you want to SEE)

The POC does **not** try to auto-fix everything. It **cleans two rules** where a correction is safe and mechanical, and **explicitly skips** the rest (recording *why*), so you can watch the difference. Every change keeps the original value alongside the new one, so it is auditable and can feed writeback.

Cleaned (auto-corrected — safe, mechanical):

- **CAM-006 CurrencyIsoCode** → `UPPER(LTRIM(RTRIM(...)))` ('gbp ' → 'GBP').
- **CAM-004 Status** → `LOWER(LTRIM(RTRIM(...)))` (case/whitespace only; unknown values stay flagged).
- **IsActive** → normalize boolean text to 0/1.
- **CAM-015 Year_c** → when blank/invalid, derive from `YEAR(StartDate)` (mark for business confirm).

Pick which checks to promote to fixes from the **rule table at the end of this stage** — it labels every rule *technical* vs *business*. Only the *technical* / *mechanical* ones belong here; *business* ones stay SKIPPED and go to writeback.

Skipped (NOT auto-changed — recorded for review):

- CAM-003 blank **Name**, CAM-005 start>end, CAM-010 missing parent, CAM-008/013 rollups, CAM-016 region, and the rest: cannot be safely auto-fixed, so they are tagged **SKIPPED** with a reason and routed to review/writeback.

Concrete build (this is `campaign_clean_build.sql`, to be created — runs as an ADF Script activity, only after Gate 3 passed):

```
-- Idempotent: rebuild each run.
DROP TABLE IF EXISTS staging.campaign_clean;

SELECT
    s.Id,
    s.SystemModstamp,
    -- CLEANED #1: currency normalized (CAM-006)
    UPPER(LTRIM(RTRIM(s.CurrencyIsoCode))) AS CurrencyIsoCode,
    s.CurrencyIsoCode AS CurrencyIsoCode_original,
    -- CLEANED #2: status case/whitespace (CAM-004)
    LOWER(LTRIM(RTRIM(s.Status))) AS Status,
    s.Status AS Status_original,
    -- CLEANED #3: IsActive boolean -> 0/1
    CASE WHEN LOWER(LTRIM(RTRIM(COALESCE(s.IsActive, '')))) IN ('true', '1', 'yes', 'y') THEN 1
         WHEN LOWER(LTRIM(RTRIM(COALESCE(s.IsActive, '')))) IN ('false', '0', 'no', 'n') THEN 0
         ELSE NULL END AS IsActive,
```

```

s.IsActive                                     AS IsActive_original,
-- CLEANED #4: Year derived from StartDate when blank/invalid (CAM-015) -- business-confirm
COALESCE(
    CASE WHEN TRY_CONVERT(int,s.Year__c) BETWEEN 2000 AND YEAR(GETDATE())+1
        THEN TRY_CONVERT(int,s.Year__c) END,
    YEAR(TRY_CONVERT(date,s.StartDate)))        AS Year__c,
s.Year__c                                     AS Year__c_original,
-- carried unchanged (skipped fields keep raw value)
s.Name, s.StartDate, s.EndDate, s.ParentId,
-- per-row clean audit (binary collation so case/whitespace-only changes are detected)
CASE WHEN s.CurrencyIsoCode COLLATE Latin1_General_BIN2 <>
UPPER(LTRIM(RTRIM(s.CurrencyIsoCode))) COLLATE Latin1_General_BIN2 THEN 'CLEANED' ELSE 'OK' END
AS clean_currency,
CASE WHEN s.Status COLLATE Latin1_General_BIN2 <> LOWER(LTRIM(RTRIM(s.Status))) COLLATE
Latin1_General_BIN2 THEN 'CLEANED' ELSE 'OK' END AS clean_status,
CASE
    WHEN s.Name IS NULL OR LTRIM(RTRIM(s.Name))='' THEN 'SKIPPED: blank
name (CAM-003)'
    WHEN TRY_CONVERT(date,s.StartDate) > TRY_CONVERT(date,s.EndDate) THEN 'SKIPPED:
start>end (CAM-005)'
    WHEN LOWER(LTRIM(RTRIM(s.Status))) NOT IN
        ('active','planned','inactive','completed','aborted','in progress') THEN 'SKIPPED:
status value needs business rule (CAM-004)'
    ELSE 'NONE'
END AS review_reason
INTO staging.campaign_clean
FROM staging.campaign_latest s;

```

See exactly what got cleaned vs skipped (this is the demonstration output):

```

SELECT 'currency_cleaned' AS metric, COUNT(*) n FROM staging.campaign_clean WHERE
clean_currency='CLEANED'
UNION ALL SELECT 'status_cleaned', COUNT(*) FROM staging.campaign_clean WHERE
clean_status='CLEANED'
UNION ALL SELECT 'skipped_rows', COUNT(*) FROM staging.campaign_clean WHERE review_reason
LIKE 'SKIPPED%';

```

Final gate on clean (both must return 0):

```

-- unique per Id (inherited from staging)
SELECT COUNT(*) FROM (SELECT Id FROM staging.campaign_clean GROUP BY Id HAVING COUNT(*)>1) x;
-- no critical-failing Id leaked into clean
SELECT COUNT(*) FROM staging.campaign_clean c
WHERE EXISTS (SELECT 1 FROM dq.dq_exceptions e
    WHERE e.object_name='Campaign' AND e.severity='CRITICAL'
    AND e.record_id = CONVERT(VARCHAR(18), c.Id));

```

Every Campaign rule — business vs technical (pick what to auto-fix)

How to read this: **Technical (auto)** = safe, mechanical — apply it on the clean table above. **Business** = needs a stakeholder decision — keep report-only or queue as approved writeback, never silently change. **Integrity/load** = can't be fixed until related data is loaded. Every rule has a fix recipe even if it currently has 0 failures, so it's ready if it ever fails.

Rule	Sev	Checks	Type	How to fix
CAM-001	CRITICAL	Id not null	Technical (gate)	Can't invent a key — hold the row out of clean and re-extract at source.
CAM-002	CRITICAL	Id is 15/18-char	Technical (gate)	Malformed id = bad extract; re-pull the record. Never auto-edit keys.
CAM-003	HIGH	Name not null	Business	Someone must supply the Name; queue for source correction, no safe default.
CAM-004	REPORT-ONLY	Status value	Technical (case) + Business (value)	Trim/case in clean; unknown values → business maps to canonical (report-only).
CAM-005	HIGH	StartDate ≤ EndDate	Business	Which date is wrong is a decision / migration artifact; don't auto-swap — review.
CAM-006	REPORT-ONLY	CurrencyIsoCode value	Technical (case) + Business (value)	Trim/upper in clean; unknown ISO codes → business confirms.
CAM-007	MEDIUM	BudgetedCost ≥ 0	Business	Coerce non-numeric → NULL (technical); a negative budget's meaning is a business call.
CAM-008	MEDIUM	AmountWon ≤ AmountAll	Business (SF rollup)	These are Salesforce roll-up fields — fix in Salesforce, not the warehouse. Report only.
CAM-009	MEDIUM	Completed/Aborted ⇒ IsActive =false	Business → writeback	Mechanically set IsActive =false to match status, but that's a correction — queue as approved writeback.
CAM-010	HIGH	ParentId exists	Integrity/load	Needs the parent Campaign loaded; if truly missing, business decides orphan vs bad ref.
CAM-011	LOW	Past EndDate ⇒ IsActive =false	Business → writeback	Same shape as CAM-009: propose IsActive =false for expired; approve before push.
CAM-012	LOW	ActualCost ≤ 200% Budget	Business	Confirm the 200% tolerance; then report-only, no auto-change.
CAM-013	MEDIUM	Opps ≤ Hierarchy opps	Business (SF rollup)	Salesforce roll-up integrity; fix in Salesforce. Report only.
CAM-014	LOW	Casesafe_Campaign_ID__c == Id	Technical (auto)	The 18-char case-safe id is deterministic from the 15-char Id — recompute it with a helper. Safe auto-fix.
CAM-015	LOW	Year__c valid 4-digit	Technical / Business	Derive from YEAR(StartDate) when blank/invalid (in clean); confirm the rule with business.
CAM-016	MEDIUM	Region__c in list	Business (governed)	Reads staging.campaign_region_allowed_values ; business owns the list. Report only.
CAM-017	HIGH	IsDeleted boolean token	Technical (auto)	Normalize the token to 0/1 (in clean); deleted rows are already filtered out of staging.
CAM-URL-001	MEDIUM	Fundraising_page_url__c pattern	Business (light technical)	Trimming is safe; adding a scheme or validating a link is a business/data decision — don't rewrite URLs silently.

What to apply on the clean table (technical/mechanical only): CAM-004 (case), CAM-006 (case), CAM-015 (Year from StartDate), CAM-017 / **IsActive** (boolean → 0/1), and CAM-014 (recompute case-safe id via a helper). **Everything else stays report-only or becomes an approved writeback** — including the "clean" rollup/date/threshold rules (CAM-005/007/008/012/013), because their *values* are business decisions, not formatting.

Stage 5 — Push-ready / writeback (NOT built yet)

The two CLEANED columns become the approved corrections queued in `writeback.campaign_pending` (Id, field, **old value** - > **new value**, rule, status). SKIPPED rows are **not** queued. The push back to Salesforce is a separate, approval-gated ADF/Function step (future). `campaign_writeback_queue.sql` still needs to be created.

Production note — container Python vs ADF

Both ingest paths are proven in Stage 1 (Path A = ADF Copy, Path B = container `python.py`). For production the plan runs the container `python.py` under **Apache NiFi** rather than a manually-triggered Container App Job; see [azure_migration_plan.md](#) (§4.2 / §4.3).

Run order summary (fully on Azure, portal)

#	Stage	Where it runs (portal)	Gate
A	Provision	Portal create blades	resources exist
B	Deploy DDL	SQL Query editor (or Cloud Shell)	objects exist on Azure SQL
1	Ingest	Path A ADF <code>pl_campaign_ingest</code> and Path B container <code>python.py</code> job — both into raw	both Succeeded , equal raw counts
2	Stage	ADF Script activity <code>EXEC ctl.refresh_latest_views</code>	0 duplicate Ids
3	Check	ADF Stored Procedure <code>dq.run_incremental_catalog_rules</code>	0 critical
4	Clean	ADF Script <code>campaign_clean_build.sql</code> (4 mechanical fixes, rest skipped)	unique + no critical leak
5	Push-ready	ADF Script <code>campaign_writeback_queue.sql</code> (to create)	approved only

Net result: Salesforce -> (ADF Copy **or** container `python.py`) -> Azure SQL (raw -> staging -> dq -> clean) -> writeback queue, all orchestrated and monitored in Azure. **Nothing runs on your PC.**

Part F — Definition Of Done (Campaign POC)

- Raw contains history/duplicates; distinct-id < row-count is accepted.
- Both ingest paths tested:** Path A (ADF Copy) and Path B (container `python.py`) each land into the same `raw.salesforce_campaign` with equal counts; Path B also proves Full / Incremental / ResumeMissing.
- `staging.campaign_latest` has **0 duplicate Ids** and excludes deleted rows.
- DQ runs incrementally and reports **0 CRITICAL** for the in-scope set.
- `staging.campaign_clean` exists, is unique per **Id**, and contains **no** critical-failing record.
- Clean **actually corrects 4 checks** (CAM-006 currency, CAM-004 status case/whitespace, IsActive boolean, CAM-015 Year-from-StartDate) and **skips the rest with a recorded reason** — the cleaned/skipped counts are visible in the demonstration query.
- The pipeline can be stopped and resumed at any stage, and state is provable from control tables.

Out of scope for this POC (needs their support): the actual **write-back / push to Salesforce is NOT required** to call this POC done. Corrections stay queued in `writeback.campaign_pending` (Stage 5); the push-back is a later, approval-gated step that needs **Salesforce admin support** (integration user + write access on the Connected App) and **business sign-off**.

Once Campaign passes, repeat the same five stages for the other 8 objects (largest last), reusing the Azure services chosen in [azure_migration_plan.md](#).

Part G — Optional Cost Estimate (3-Day POC)

Scenario: **Campaign only** (~41K rows / ~50 MB), UK South, over **3 days** — one **full load**, a few **incrementals** per day, running the **rules**, and building the **clean** table. This is a relative estimate, **not a quote** — confirm with the Azure Pricing Calculator.

Service	What drives cost	3-day estimate
Azure SQL Database (serverless, GP 1 vCore, auto-pause 1h)	vCore-seconds while active + ~5 GB storage	~£4–6 (near £0 while paused)
Azure Data Factory	activity-run count + Copy DIU-hours (tiny dataset)	~£1–2
ADLS Gen2	50 MB + a few thousand operations	< £0.10
Key Vault	per-secret operation	< £0.05
Log Analytics (if enabled for logs)	ingested log volume (small)	< £0.20
Total		~£6–10 (~\$8–13)

Keep it low: leave SQL **auto-pause** on (idles to near-zero), run pipelines **on demand** (not a tight schedule), and **delete the resource group** when done. Serverless compute dominates the bill; everything else is pennies at this size. Cost scales with the *big* objects later (Opportunity, Item Allocation) — not with Campaign.## Part G — Optional Cost Estimate (3-Day POC)

Scenario: **Campaign only** (~41K rows / ~50 MB), over **3 days** — one **full load**, a few **incrementals** per day, running the **rules**, and building the **clean** table. Relative estimate in **USD**, **not a quote** — confirm with the [Azure Pricing Calculator](#).

Service	Why it costs (billing driver)	3-day estimate (USD)
Azure SQL Database (serverless, GP 1 vCore, auto-pause 1h)	billed per vCore-second while active + ~5 GB storage; pauses to ~\$0 when idle	~\$5–8
Azure Data Factory	per activity run + Copy DIU-hour (tiny dataset)	~\$1.50–3
ADLS Gen2	50 MB storage + a few thousand operations	< \$0.15
Key Vault	per-secret operation	< \$0.05
Log Analytics (if logs enabled)	per GB ingested (small)	< \$0.25
Total		~\$8–13

Why so low: serverless SQL auto-pauses (near-\$0 idle) and Campaign is tiny, so vCore-seconds and DIU-hours are minimal.

Pricing references: [Azure SQL Database serverless](#) · [Data Factory pipelines](#) · [ADLS Gen2 / Blob](#) · [Key Vault](#) · [Azure Monitor / Log Analytics](#).